

Algorithm for file updates in Python

Project description

In this project, I developed a Python algorithm to automate security access control updates. The scenario involves managing an "allow list" text file (allow_list.txt) that contains authorized IP addresses. Due to policy updates, certain IP addresses identified in a separate list (remove_list) need to be revoked and removed from this system file. The objective of the program is to read the original file, dynamically filter out the restricted IP addresses, and overwrite the file with the updated list of permitted addresses.

Open the file that contains the allow list

To begin processing, I first needed to open the allow_list.txt file in read mode. I achieved this by using the with statement along with the open() function and the "r" parameter. This approach ensures that the file automatically closes properly after its contents are accessed, preventing memory leaks or resource locks.

```
python
```

```
import_file = "allow_list.txt"
```

```
# Opening the file in read-only mode
```

```
with open(import_file, "r") as file:
```

Read the file contents

Once the file was open, I extracted its content into a single string variable named ip_addresses. This was completed by applying the .read() method directly to the file object created within the with block. This method reads all the text inside the file as a continuous string.

```
python
```

```
    # Reading the contents and storing them in a variable
```

```
    ip_addresses = file.read()
```

Convert the string into a list

Because the file contents were loaded as a single string, I could not easily target and remove individual IP addresses. To convert the text into an iterative format, I used the .split() method. By

default, `.split()` breaks a string down at every whitespace character (like new lines or spaces), reorganizing the text into a clean Python list of IP addresses.

```
python
```

```
# Converting the long text string into a list of individual IP strings
```

```
ip_addresses = ip_addresses.split()
```

Iterate through the remove list

[To evaluate which IP addresses needed to be removed, I constructed a conditional loop. By utilizing a for loop, the algorithm cycles through every single element inside the `remove_list` variable one by one, allowing us to perform a comparison checks on each target IP.

```
python
```

```
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]
```

```
# Starting a loop to inspect every restricted IP
```

```
for element in remove_list:
```

Remove IP addresses that are on the remove list

Inside the loop body, I nested an if conditional statement to see if any restricted IP (element) currently existed within our main `ip_addresses` list. If the condition evaluated to True, the algorithm triggered the `.remove()` method on the main list to strip out that specific unauthorized entry.

```
python
```

```
    # Checking if the restricted IP exists in the allow list
```

```
    if element in ip_addresses:
```

```
        # Erasing the restricted IP from the list
```

```
        ip_addresses.remove(element)
```

Update the file with the revised list of IP addresses

After removing all restricted IPs, the data was still in a list format, but files require a single string to save. I converted the list back into a string using the `".join(ip_addresses)` method, which stitches the list items together separated by spaces. Finally, I opened the original file again—this time using the "w" (write) parameter—and applied the `.write()` method to completely overwrite the old content with our new, sanitized IP address string.

```
python
```

```
# Rejoining the list back into a clean string format
```

```
ip_addresses = " ".join(ip_addresses)
```

```
# Opening the file in write mode to rewrite its contents  
with open(import_file, "w") as file:  
    file.write(ip_addresses)
```

Summary

In summary, I successfully constructed a Python script that reads a text log, parses it into an editable list, filters out unauthorized IP addresses using a for loop combined with an if condition, and rewrites the updated data cleanly back into the server configuration file. This project highlights how basic file handling (.read(), .write()), string manipulation (.split(), .join()), and control structures can be combined to solve real-world automation tasks in cybersecurity.